

DBMS Assignment 1

By Himanshu Sardana

Question 1

Create a table Student (Rno, Name, DOB, Gender, Class, College, City, Marks)

```
CREATE TABLE Student (  
    RNo NUMBER(10),  
    Name VARCHAR(255),  
    DOB DATE,  
    Gender CHAR(1),  
    Class VARCHAR(5),  
    College VARCHAR(50),  
    City VARCHAR(25),  
    Marks NUMBER(3)  
);
```

Output:

```
Table created.
```

Question 2

Insert 5 records into the Student table

```
INSERT INTO Student VALUES (102303244, 'Himanshu Sardana', '04-JUN-2005', 'M', '2C21',  
'Thapar', 'Patiala', 25);  
INSERT INTO Student VALUES (102303243, 'Harith Khanna', '03-JUL-2005', 'M', '2C21',  
'Thapar', 'Patiala', 98);  
INSERT INTO Student VALUES (102303239, 'Nitish', '02-JAN-2005', 'M', '2C21', 'Thapar',  
'Amritsar', 96);  
INSERT INTO Student VALUES (102303241, 'Angad Dhawan', '01-JAN-2005', 'M', '2C21', 'Thapar',  
'Patiala', 100);  
INSERT INTO Student VALUES (102303242, 'Arnav Goyal', '05-MAR-2004', 'M', '2C21', 'Thapar',  
'Patiala', 96);
```

Output:

```
1 row(s) inserted.  
1 row(s) inserted.  
1 row(s) inserted.  
1 row(s) inserted.  
1 row(s) inserted.
```

Question 3

Display the information of all the students

```
SELECT * FROM Student;
```

Output:

RNO	NAME	DOB	Gender	Class	College	City	Marks
102303244	Himanshu Sardana	04-JUN-2005	M	2C21	Thapar	Patiala	25
102303243	Harith Khanna	03-JUL-2005	M	2C21	Thapar	Patiala	98
102303239	Nitish	02-JAN-2005	M	2C21	Thapar	Amritsar	96
102303241	Angad Dhawan	01-JAN-2005	M	2C21	Thapar	Patiala	100
102303242	Arnav Goyal	05-MAR-2004	M	2C21	Thapar	Patiala	96

Question 4

Display the detailed structure of the Student table

```
DESCRIBE Student;
```

Output:

Column	Null?	Type
RNO	-	NUMBER(10, 0)
NAME	-	VARCHAR2(255)
DOB	-	DATE
Gender	-	CHAR(1)
Class	-	VARCHAR2(5)
College	-	VARCHAR2(50)
City	-	VARCHAR2(25)
Marks	-	NUMBER(3, 0)

Question 5

Display Rno, Name and Class information of 'Patiala' students

```
SELECT RNo, Name, Class FROM Student WHERE City = 'Patiala'
```

Output:

RNO	NAME	Class
102303244	Himanshu Sardana	2C21
102303243	Harith Khanna	2C21
102303241	Angad Dhawan	2C21
102303242	Arnav Goyal	2C21

Question 6

Display information on ascending order of marks

```
SELECT * FROM Student ORDER BY Marks;
```

Output:

RNO	NAME	DOB	Gender	Class	College	City	Marks
102303244	Himanshu Sardana	04-JUN-2005	M	2C21	Thapar	Patiala	25
102303243	Harith Khanna	03-JUL-2005	M	2C21	Thapar	Patiala	98
102303239	Nitish	02-JAN-2005	M	2C21	Thapar	Amritsar	96
102303242	Arnav Goyal	05-MAR-2004	M	2C21	Thapar	Patiala	96
102303241	Angad Dhawan	01-JAN-2005	M	2C21	Thapar	Patiala	100

Question 7

Change the marks of Rno 5 to 89

```
UPDATE Student SET Marks = 89 WHERE RNo = '102303243';
```

Output:

```
1 row(s) updated.
```

Question 8

Change the name and city of Rno 9

```
UPDATE Student SET Marks = 90, CITY = 'Delhi' WHERE RNo = '102303244';
```

Output:

```
1 row(s) updated.
```

Question 9

Delete the information of 'Amritsar' city records.

```
DELETE FROM Student WHERE CITY = 'Amritsar';
```

Output:

```
1 row(s) deleted.
```

Question 10

Delete the records of students where marks < 30.

```
DELETE FROM STUDENT WHERE MARKS < 30;
```

Output:

```
1 row(s) deleted.
```

DBMS Assignment 2

By Himanshu Sardana

```
CREATE TABLE Emp (  
    EmpNo NUMBER(10),  
    EName VARCHAR(255),  
    Job VARCHAR(25),  
    Salary NUMBER(7),  
    Commission NUMBER(7),  
    DeptNo NUMBER(3)  
);  
  
INSERT INTO Emp VALUES (1, 'Himanshu', 'IT Manager', 10000, 1000, 1);  
INSERT INTO Emp VALUES (1, 'Nitish', 'Manager', 20000, NULL, 15);  
INSERT INTO Emp VALUES (3, 'Angad', 'CEO', 999999, NULL, 10);  
INSERT INTO Emp VALUES (4, 'Arnav', 'Clerk', 100000, 10000, 12);  
INSERT INTO Emp VALUES (5, 'Harith Khanna', 'Clerk', 120000, 12000, 12);  
  
SELECT EmpNo, EName FROM Emp WHERE DeptNo = 10;  
  
SELECT EName FROM Emp WHERE Job = 'Clerk' AND Salary > 2000;  
  
SELECT EName, Job FROM Emp WHERE Job IN ('Salesperson', 'Clerk');  
  
SELECT * FROM Emp WHERE Salary BETWEEN 2000 AND 3000;  
  
SELECT * FROM Emp WHERE DeptNo IN (10,20,30);  
  
SELECT Ename FROM Emp WHERE Commission IS NULL;  
  
SELECT DeptNo, Salary FROM Emp ORDER BY DeptNo ASC, SALARY DESC;  
  
SELECT EName FROM Emp WHERE EName LIKE '%a%a%' OR EName LIKE '%A%A%';  
  
SELECT EName FROM Emp WHERE EName LIKE '_b%' OR EName LIKE '_B%';  
  
SELECT EName FROM Emp WHERE EName LIKE 'a%' OR EName LIKE 'A%' OR EName LIKE '%A' OR EName  
LIKE '%a';  
  
SELECT MAX(Salary), MIN(Salary), AVG(Salary) FROM Emp WHERE DeptNo = 10;  
  
SELECT COUNT(*) FROM Emp WHERE DeptNo = 20;  
  
SELECT SUM(Salary) FROM Emp WHERE Job = 'Clerk';  
  
SELECT CURRENT_DATE FROM Emp;  
  
SELECT (12*12/13) FROM Emp;  
  
SELECT * FROM Emp WHERE UPPER(Ename) = UPPER('raj');
```


DBMS Assignment 3

By Himanshu Sardana

Q1: Use the following functions:

- chr(n)

```
SELECT chr(65) FROM dual;
```

OUTPUT

CHR(65)

A

- concat(char1, char2)

```
SELECT concat('Himanshu', ' ', 'Sardana') FROM dual;
```

OUTPUT

CONCAT('HIMANSHU', ' ', 'SARDANA')

Himanshu Sardana

- instr(string, char)

```
SELECT instr('Himanshu', 'a') FROM dual;
```

OUTPUT

INSTR('HIMANSHU', 'A')

3

- length(n)

```
SELECT length('Himanshu') FROM dual;
```

OUTPUT

LENGTH('HIMANSHU')

8

- lpad (string, length, char)

```
SELECT lpad('Himanshu', 10, '**') FROM dual;
```

OUTPUT

```
| LPAD('HIMANSHU',10,'**') |  
|-----|  
| **Himanshu              |
```

- ltrim(string, char)

```
SELECT ltrim('Himanshu', 'H') FROM dual;
```

OUTPUT

```
| LTRIM('HIMANSHU', 'H') |  
|-----|  
| imanshu                 |
```

- rpad(string, length, char)

```
SELECT rpad('Himanshu', 10, '**') FROM dual;
```

OUTPUT

```
| RPAD('HIMANSHU',10,'**') |  
|-----|  
| Himanshu**              |
```

- rtrim(string, char)

```
SELECT rtrim('Himanshu', 'u') FROM dual;
```

OUTPUT

```
| RTRIM('HIMANSHU', 'U') |  
|-----|  
| Himansh                 |
```

- replace(char, search_string, replace_string)

```
SELECT replace('Himanshu', 'a', 'A') FROM dual;
```

OUTPUT

```
| REPLACE('HIMANSHU', 'A', 'A') |  
|-----|  
| HimAnshu                      |
```


- substr(string, start, length)

```
SELECT substr('Himanshu', 2, 4) FROM dual;
```

OUTPUT

```
| SUBSTR('HIMANSHU',2,4) |  
|-----|  
| iman                  |
```

- initcap(char)

```
SELECT initcap('himanshu sardana') FROM dual;
```

OUTPUT

```
| INITCAP('HIMANSHUSARDANA') |  
|-----|  
| Himanshu Sardana          |
```

- lower(char)

```
SELECT lower('HIMANSHU') FROM dual;
```

OUTPUT

```
| LOWER('HIMANSHU') |  
|-----|  
| himanshu          |
```

- upper(char)

```
SELECT upper('himanshu') FROM dual;
```

OUTPUT

```
| UPPER('HIMANSHU') |  
|-----|  
| HIMANSHU          |
```

- translate(char, search_string, replace_string)

```
SELECT translate('Himanshu', 'H', 'h') FROM dual;
```

OUTPUT

```
| TRANSLATE('HIMANSHU','H','h') |
|-----|
| himanshu                      |
```

- `abs(n)`

```
SELECT abs(-10) FROM dual;
```

OUTPUT

```
| ABS(-10) |
|-----|
| 10       |
```

- `ceil(n)`

```
SELECT ceil(10.1) FROM dual;
```

OUTPUT

```
| CEIL(10.1) |
|-----|
| 11         |
```

- `cos(n)`

```
SELECT cos(0) FROM dual;
```

OUTPUT

```
| COS(0) |
|-----|
| 1      |
```

- `exp(n)`

```
SELECT exp(1) FROM dual;
```

OUTPUT

```
| EXP(1) |
|-----|
| 2.7183 |
```

- `floor(n)`

```
SELECT floor(10.1) FROM dual;
```

OUTPUT

	FLOOR(10.1)	

	10	

- mod(m, n)

```
SELECT mod(10, 3) FROM dual;
```

OUTPUT

	MOD(10, 3)	

	1	

- power(m, n)

```
SELECT power(2, 3) FROM dual;
```

OUTPUT

	POWER(2, 3)	

	8	

- round(n)

```
SELECT round(10.1) FROM dual;
```

OUTPUT

	ROUND(10.1)	

	10	

- sign(n)

```
SELECT sign(-10) FROM dual;
```

OUTPUT

	SIGN(-10)	

	-1	

- sqrt(n)

```
SELECT sqrt(9) FROM dual;
```

OUTPUT

```
| SQRT(9) |  
|-----|  
| 3      |
```

- trunc(x, n)

```
SELECT trunc(10.123, 2) FROM dual;
```

OUTPUT

```
| TRUNC(10.123,2) |  
|-----|  
| 10.12          |
```

- sysdate;

```
SELECT sysdate FROM dual;
```

OUTPUT

```
22-JAN-25
```

- greatest(expr)

```
SELECT greatest(10, 20, 30) FROM dual;
```

OUTPUT

```
| GREATEST(10,20,30) |  
|-----|  
| 30                 |
```

- least(expr)

```
SELECT least(10, 20, 30) FROM dual;
```

OUTPUT

```
| LEAST(10,20,30) |  
|-----|  
| 10              |
```

Question 2

- Display current time in hour : min : sec format

```
SELECT to_char(sysdate, 'HH24:MI:SS') FROM dual;
```

OUTPUT

```
03:20:33
```

DBMS Assignment 4

By Himanshu Sardana

Question 1

Display the system date

```
SELECT SYSDATE FROM DUAL;
```

Output:

```
SYSDATE
-----
29-JAN-25
```

Question 2

Display current day

```
SELECT TO_CHAR(SYSDATE, 'DAY') FROM DUAL;
```

Output:

```
TO_CHAR(SYSDATE, 'DAY')
-----
WEDNESDAY
```

Question 3

Display current month and spell out year

```
SELECT TO_CHAR(SYSDATE, 'MONTH') || ' ' || TO_CHAR(SYSDATE, 'YEAR') FROM DUAL;
```

```
TO_CHAR(SYSDATE, 'MONTH') || ' ' || TO_CHAR(SYSDATE, 'YYYY')
-----
JANUARY TWENTY TWENTY FIVE
```

Question 4

Display spell out current date

```
SELECT TO_CHAR(SYSDATE, 'DAY') || ' ' || TO_CHAR(SYSDATE, 'MONTH') || ' ' ||
TO_CHAR(SYSDATE, 'YYYY') FROM DUAL;
```

Output:

```
TO_CHAR(SYSDATE, 'DD') || ' ' || TO_CHAR(SYSDATE, 'MONTH') || ' ' || TO_CHAR(SYSDATE, 'YYYY')
-----
TWENTY NINE JANUARY TWENTY TWENTY FIVE
```

Question 5

Check whether it is AM or PM right now

```
SELECT TO_CHAR(SYSDATE, 'AM') FROM DUAL;
```

Output:

```
TO_CHAR(SYSDATE, 'AM')
-----
AM
```

Question 6

Display the date of next friday

```
SELECT NEXT_DAY(SYSDATE, 'FRIDAY') FROM DUAL;
```

Output:

```
NEXT_DAY(SYSDATE, 'FRIDAY')
-----
07-FEB-25
```

Question 7

Round the system date on month

```
SELECT ROUND(SYSDATE, 'MM') FROM DUAL;
```

Output:

```
ROUND(SYSDATE, 'MM')
-----
29-JAN-25
```

Question 8

Truncate the system date on month

```
SELECT TRUNC(SYSDATE, 'MM') FROM DUAL;
```

Output:

```
TRUNC (SYSDATE, 'MM')
```

```
-----
```

```
29-JAN-25
```

Question 9

Round the system date on year

```
SELECT ROUND (SYSDATE, 'YEAR') FROM DUAL;
```

Output:

```
ROUND (SYSDATE, 'YEAR')
```

```
-----
```

```
01-JAN-26
```

Question 10

Truncate the system date on year

```
SELECT TRUNC (SYSDATE, 'YEAR') FROM DUAL;
```

Output:

```
TRUNC (SYSDATE, 'YEAR')
```

```
-----
```

```
01-JAN-25
```

Question 11

Find the day after three days

```
SELECT SYSDATE + 3 FROM DUAL;
```

Output:

```
SYSDATE+3
```

```
-----
```

```
01-FEB-25
```

```
CREATE TABLE Emp (  
    EmpNo NUMBER(2),  
    Name VARCHAR(20),  
    date_of_joining DATE,  
);
```


Question 12

Create an Emp table (EmpNo Number, Name VARCHAR(20), date_of_joining DATE)

```
CREATE TABLE Emp (  
    EmpNo NUMBER(2),  
    Name VARCHAR(20),  
    date_of_joining DATE  
);
```

Question 13

Display date_of_joining column

```
SELECT TO_CHAR(date_of_joining, 'dd-mm-yy') "Date of Joining" FROM Emp;
```

Question 14

Display those employees whose date of joining is on a monday

```
SELECT * FROM Emp WHERE TO_CHAR(date_of_joining, 'DAY') = 'MONDAY';
```

Question 15

Display those employees who joined the company this month

```
SELECT * FROM Emp WHERE TO_CHAR(date_of_joining, 'MM') = TO_CHAR(SYSDATE, 'MM');
```

Question 16

Display those employees who joined the company in the last 30 days

```
SELECT * FROM Emp WHERE date_of_joining > SYSDATE - 30;
```

Question 17

Create a table Train having four columns (TrainNo NUMBER(6), date_of_departure DATE, time_of_departure timestamp, time_of_arrival timestamp)

```
CREATE TABLE Train (  
    TrainNo NUMBER(6),  
    date_of_departure DATE,  
    time_of_departure TIMESTAMP,  
    time_of_arrival TIMESTAMP  
);
```

Question 18

Insert five records into the train table

```
INSERT INTO Train VALUES (123456, '29-JAN-25', '29-JAN-25 10:00:00', '29-JAN-25 12:00:00');
INSERT INTO Train VALUES (123457, '29-JAN-25', '29-JAN-25 11:00:00', '29-JAN-25 13:00:00');
INSERT INTO Train VALUES (123458, '29-JAN-25', '29-JAN-25 12:00:00', '29-JAN-25 14:00:00');
INSERT INTO Train VALUES (123459, '29-JAN-25', '29-JAN-25 13:00:00', '29-JAN-25 15:00:00');
INSERT INTO Train VALUES (123460, '29-JAN-25', '29-JAN-25 14:00:00', '29-JAN-25 16:00:00');
```

Question 19

Display all the records

```
SELECT * FROM Train;
```

Question 20

Display the time values inserted in the columns

```
SELECT TO_CHAR(time_of_departure, 'HH24:MI:SS') "Time of Departure",
TO_CHAR(time_of_arrival, 'HH24:MI:SS') "Time of Arrival" FROM Train;
```

Question 21

Display those trains which arrived on PM

```
SELECT * FROM Train WHERE TO_CHAR(time_of_arrival, 'AM') = 'PM';
```

Question 22

Display train number that is going to depart in next one hour

```
SELECT * FROM Train WHERE time_of_departure < SYSDATE + 1/24;
```

DBMS Assignment 5

By Himanshu Sardana

Question 1

Create table dept which has the following attributes (department table) (deptno, dept_name) where deptno is primary key and dept_name in (Acc, comp, elect)

```
CREATE TABLE dept (  
    deptno INT PRIMARY KEY,  
    dept_name VARCHAR(10) CHECK (dept_name IN ('Acc', 'comp', 'elect'))  
);
```

Question 2

Create table emp which has the following attributes (empno, emp_name, job, sal, deptno, mgr_no) where empno is primary key, emp_no is primary key, emp_name is unique, job in (Prof, AP and Lect), sal is not null, dept_no is foreign key, mgr_no is self referential foreign key.

```
CREATE TABLE emp (  
    empno INT PRIMARY KEY,  
    emp_name VARCHAR(20) UNIQUE,  
    job VARCHAR(10) CHECK (job IN ('Prof', 'AP', 'Lect')),  
    sal INT NOT NULL,  
    deptno INT,  
    mgr_no INT,  
    FOREIGN KEY (deptno) REFERENCES dept(deptno),  
    FOREIGN KEY (mgr_no) REFERENCES emp(empno)  
);
```

Question 3

Create a table **S** which has the following attributes (Salesperson table) (sno, sname, city) where sno is primary key.

```
CREATE TABLE S (  
    sno INT PRIMARY KEY,  
    sname VARCHAR(20),  
    city VARCHAR(20)  
);
```

Question 4

Create a table **P** which has the following attributes (Part table) (pno, pname, color) where pno is primary key.

```
CREATE TABLE P (  
    pno INT PRIMARY KEY,  
    pname VARCHAR(20),  
    color VARCHAR(20)  
);
```

Question 5

Create a table **J** which has the following attributes (Project Table) (jno, jname, city) where jno is primary key.

```
CREATE TABLE J (  
    jno INT PRIMARY KEY,  
    jname VARCHAR(20),  
    city VARCHAR(20)  
);
```

Question 6

Create a table **SPJ** which has the following attributes (sno, pno, jno, qty) where combination of (sno, pno, jno) is a composite primary key, Also, sno, pno, jno are foreign keys.

```
CREATE TABLE SPJ (  
    sno INT,  
    pno INT,  
    jno INT,  
    qty INT,  
    PRIMARY KEY (sno, pno, jno),  
    FOREIGN KEY (sno) REFERENCES S(sno),  
    FOREIGN KEY (pno) REFERENCES P(pno),  
    FOREIGN KEY (jno) REFERENCES J(jno)  
);
```

Question 7

Insert atleast 5 appropriate records in the above tables

```
INSERT INTO dept VALUES (1, 'Acc');  
INSERT INTO dept VALUES (2, 'comp');  
INSERT INTO dept VALUES (3, 'elect');  
INSERT INTO dept VALUES (4, 'mech');  
  
INSERT INTO emp VALUES (1, 'Robert', 'Prof', 10000, 1, 1);  
INSERT INTO emp VALUES (2, 'John', 'AP', 8000, 2, 1);  
INSERT INTO emp VALUES (3, 'Doe', 'Lect', 6000, 3, 1);  
INSERT INTO emp VALUES (4, 'Jane', 'Prof', 10000, 1, 2);  
INSERT INTO emp VALUES (5, 'Arthur', 'AP', 8000, 2, 2);
```

```

INSERT INTO S VALUES (1, 'Robert', 'Delhi');
INSERT INTO S VALUES (2, 'John', 'Mumbai');
INSERT INTO S VALUES (3, 'Doe', 'Chennai');
INSERT INTO S VALUES (4, 'Jane', 'Kolkata');
INSERT INTO S VALUES (5, 'Arthur', 'Bangalore');

INSERT INTO P VALUES (1, 'Part1', 'Red');
INSERT INTO P VALUES (2, 'Part2', 'Blue');
INSERT INTO P VALUES (3, 'Part3', 'Green');
INSERT INTO P VALUES (4, 'Part4', 'Yellow');
INSERT INTO P VALUES (5, 'Part5', 'Black');

INSERT INTO J VALUES (1, 'Project1', 'Delhi');
INSERT INTO J VALUES (2, 'Project2', 'Mumbai');
INSERT INTO J VALUES (3, 'Project3', 'Chennai');
INSERT INTO J VALUES (4, 'Project4', 'Kolkata');
INSERT INTO J VALUES (5, 'Project5', 'Bangalore');

INSERT INTO SPJ VALUES (1, 1, 1, 10);
INSERT INTO SPJ VALUES (2, 2, 2, 20);
INSERT INTO SPJ VALUES (3, 3, 3, 30);
INSERT INTO SPJ VALUES (4, 4, 4, 40);
INSERT INTO SPJ VALUES (5, 5, 5, 50);

```

Question 8

Drop the `NOT NULL` constraints from EMP table

```
ALTER TABLE emp ALTER COLUMN sal DROP NOT NULL;
```

Question 9

Check all the constraints name and their type in the EMP table

```
DESCRIBE emp;
```

Question 10

Drop the UNIQUE constraint on EMP_NAME from the EMP table

```
ALTER TABLE emp DROP INDEX emp_name;
```

Question 11

Drop the foreign key constraint on DEPTNO from EMP table

```
ALTER TABLE emp DROP FOREIGN KEY deptno;
```

Question 12

Add Foreign Key Constraint on Deptno in EMP table

```
ALTER TABLE emp ADD FOREIGN KEY (deptno) REFERENCES dept (deptno);
```

Question 13

Drop the Check constraint from DEPT table

```
ALTER TABLE dept DROP CHECK dept_name;
```

Question 14

Add COMM column in EMP table (default value 0)

```
ALTER TABLE emp ADD COLUMN comm INT DEFAULT 0;
```

Question 15

Drop default constraint from EMP

```
ALTER TABLE emp ALTER COLUMN comm DROP DEFAULT;
```

Question 16

Create duplicate copy of EMP table

```
CREATE TABLE emp_copy AS SELECT * FROM emp;
```

Question 17

Copy the structure of Dept table to a new table with different column names

```
CREATE TABLE dept_copy AS SELECT deptno AS dept_id, dept_name AS dept_name FROM dept;
```

Question 18

Copy the structure of Dept table to a new table with different column names without any records copied from Depto

```
CREATE TABLE dept_copy AS SELECT deptno AS dept_id, dept_name AS dept_name FROM dept WHERE 1=0;
```

Question 19

Change the name and job of the employee whose EmpNo = 100

```
UPDATE emp SET emp_name = 'Robert', job = 'Prof' WHERE empno = 100;
```

Question 20

Delete the record of employee who belongs to computer department

```
DELETE FROM emp WHERE deptno = 2;
```

Question 21

Delete dept no 101 from Dept table and set NULL to the corresponding deptno in EMP table

```
DELETE FROM dept WHERE deptno = 101;  
UPDATE emp SET deptno = NULL WHERE deptno = 101;
```

Question 22

Delete dept no 102 from Dept table and its corresponding record from the EMP table

```
DELETE FROM dept WHERE deptno = 102;  
DELETE FROM emp WHERE deptno = 102;
```

Question 23

Delete from empno 111 who is the manager of the employee whose empno is 114

```
DELETE FROM emp WHERE empno = 111;  
UPDATE emp SET mgr_no = NULL WHERE mgr_no = 111;
```

Question 24

Delete the record of 'Ravi' whose empno is 112 and set mgr_no to NULL for the employees for whom Ravi is the manager

```
DELETE FROM emp WHERE empno = 112;  
UPDATE emp SET mgr_no = NULL WHERE mgr_no = 112;
```

Question 25

Drop the duplicate copy of EMP table

```
DROP TABLE emp_copy;
```

DBMS Assignment 6

By Himanshu Sardana

TABLE DEPT (DEPTNO, DEPTNAME)

TABLE EMP (EMPNO, EMPNAME, DEPTNO, JOB, SALARY): DEPTNO is a foreign

key w.r.t to DEPT Table

```
CREATE TABLE DEPT (  
    DEPTNO NUMBER(3),  
    DEPTNAME VARCHAR(255)  
);  
  
CREATE TABLE EMP (  
    EMPNO NUMBER(10),  
    EMPNAME VARCHAR(255),  
    DEPTNO NUMBER(3),  
    JOB VARCHAR(25),  
    SALARY NUMBER(7)  
    CONSTRAINT FK_DEPTNO FOREIGN KEY (DEPTNO) REFERENCES DEPT(DEPTNO)  
);
```

Question 1

Insert 10 appropriate records in each table as per the SQL queries mentioned below.

```
INSERT INTO DEPT (DEPTNO, DEPTNAME) VALUES (10, 'HR');  
INSERT INTO DEPT (DEPTNO, DEPTNAME) VALUES (20, 'IT');  
INSERT INTO DEPT (DEPTNO, DEPTNAME) VALUES (30, 'Finance');  
INSERT INTO DEPT (DEPTNO, DEPTNAME) VALUES (40, 'Marketing');  
INSERT INTO DEPT (DEPTNO, DEPTNAME) VALUES (50, 'Sales');  
INSERT INTO DEPT (DEPTNO, DEPTNAME) VALUES (60, 'Admin');  
INSERT INTO DEPT (DEPTNO, DEPTNAME) VALUES (70, 'Support');  
INSERT INTO DEPT (DEPTNO, DEPTNAME) VALUES (80, 'Research');  
INSERT INTO DEPT (DEPTNO, DEPTNAME) VALUES (90, 'Development');  
INSERT INTO DEPT (DEPTNO, DEPTNAME) VALUES (100, 'Testing');  
  
INSERT INTO EMP (EMPNO, EMPNAME, DEPTNO, JOB, SALARY) VALUES (1, 'Himanshu', 10, 'IT  
Manager', 10000);  
INSERT INTO EMP (EMPNO, EMPNAME, DEPTNO, JOB, SALARY) VALUES (2, 'Albert', 20, 'Manager',  
20000);  
INSERT INTO EMP (EMPNO, EMPNAME, DEPTNO, JOB, SALARY) VALUES (3, 'David', 30, 'CEO',  
999999);  
INSERT INTO EMP (EMPNO, EMPNAME, DEPTNO, JOB, SALARY) VALUES (4, 'Richard', 40, 'Clerk',  
100000);  
INSERT INTO EMP (EMPNO, EMPNAME, DEPTNO, JOB, SALARY) VALUES (5, 'Harry', 50, 'Clerk',  
120000);  
INSERT INTO EMP (EMPNO, EMPNAME, DEPTNO, JOB, SALARY) VALUES (6, 'John', 60, 'Manager',  
15000);
```



```
INSERT INTO EMP (EMPNO, EMPNAME, DEPTNO, JOB, SALARY) VALUES (7, 'Jane', 70, 'Analyst', 8000);
INSERT INTO EMP (EMPNO, EMPNAME, DEPTNO, JOB, SALARY) VALUES (8, 'Doe', 80, 'Developer', 6000);
INSERT INTO EMP (EMPNO, EMPNAME, DEPTNO, JOB, SALARY) VALUES (9, 'Smith', 90, 'Tester', 5000);
INSERT INTO EMP (EMPNO, EMPNAME, DEPTNO, JOB, SALARY) VALUES (10, 'Brown', 100, 'Support', 7000);
```

Question 2

List the total number of employees

```
SELECT COUNT(*) FROM EMP;
```

Question 3

List the total no of departments?

```
SELECT COUNT(*) FROM DEPT;
```

Question 4

Display the employee details of each department for which they are working.

```
SELECT EMP.EMPNO, EMP.EMPNAME, DEPT.DEPTNAME
FROM EMP
JOIN DEPT ON EMP.DEPTNO = DEPT.DEPTNO;
```

Question 5

List the total, maximum, & minimum salary for deptno 30?

```
SELECT SUM(SALARY) AS TOTAL_SALARY, MAX(SALARY) AS MAX_SALARY, MIN(SALARY) AS MIN_SALARY
FROM EMP
WHERE DEPTNO = 30;
```

Question 6

Display the name of the employee getting maximum salary?

```
SELECT EMPNAME
FROM EMP
WHERE SALARY = (SELECT MAX(SALARY) FROM EMP);
```

Question 7

Display the total salary for each department?

```
SELECT DEPT.DEPTNAME, SUM(EMP.SALARY) AS TOTAL_SALARY
FROM EMP
JOIN DEPT ON EMP.DEPTNO = DEPT.DEPTNO
GROUP BY DEPT.DEPTNAME;
```

Question 8

Display the total salary for each job.

```
SELECT JOB, SUM(SALARY) AS TOTAL_SALARY
FROM EMP
GROUP BY JOB;
```

Question 9

Display the total salary for each job within each department.

```
SELECT DEPT.DEPTNAME, EMP.JOB, SUM(EMP.SALARY) AS TOTAL_SALARY
FROM EMP
JOIN DEPT ON EMP.DEPTNO = DEPT.DEPTNO
GROUP BY DEPT.DEPTNAME, EMP.JOB;
```

Question 10

Display the average salary for each job in deptno 20.

```
SELECT JOB, AVG(SALARY) AS AVERAGE_SALARY
FROM EMP
WHERE DEPTNO = 20
GROUP BY JOB;
```

Question 11

Display the total salary for each job excluding the 'manager' and 'salesman' job

```
SELECT JOB, SUM(SALARY) AS TOTAL_SALARY
FROM EMP
WHERE JOB NOT IN ('manager', 'salesman')
GROUP BY JOB;
```

Question 12

Display the average salary for each job in deptno 20, but only display those jobs where average salary is greater than 2000 & display the output in descending order of salary?

```
SELECT JOB, AVG(SALARY) AS AVERAGE_SALARY
FROM EMP
WHERE DEPTNO = 20
GROUP BY JOB
HAVING AVG(SALARY) > 2000
ORDER BY AVERAGE_SALARY DESC;
```

Question 13

Display the total no of employees for each department excluding the deptno 10 & display only those departments where more than five employees work. Display the output in descending order of total no of employees?

```
SELECT DEPT.DEPTNAME, COUNT(EMP.EMPNO) AS TOTAL_EMPLOYEES
FROM EMP
JOIN DEPT ON EMP.DEPTNO = DEPT.DEPTNO
WHERE DEPT.DEPTNO <> 10
GROUP BY DEPT.DEPTNAME
HAVING COUNT(EMP.EMPNO) > 5
ORDER BY TOTAL_EMPLOYEES DESC;
```

Question 14

Display the total no of employees for each department excluding the 'comp' dept & display only those departments where more than five employees work. Display the output in descending order of total no of employees?

```
SELECT DEPT.DEPTNAME, COUNT(EMP.EMPNO) AS TOTAL_EMPLOYEES
FROM EMP
JOIN DEPT ON EMP.DEPTNO = DEPT.DEPTNO
WHERE DEPT.DEPTNAME <> 'comp'
GROUP BY DEPT.DEPTNAME
HAVING COUNT(EMP.EMPNO) > 5
ORDER BY TOTAL_EMPLOYEES DESC;
```

Question 15

Display total number of emp working in each job in each dept.

```
SELECT DEPT.DEPTNAME, EMP.JOB, COUNT(EMP.EMPNO) AS TOTAL_EMPLOYEES
FROM EMP
JOIN DEPT ON EMP.DEPTNO = DEPT.DEPTNO
GROUP BY DEPT.DEPTNAME, EMP.JOB;
```

Question 16

List all the department name and their employees name even if for a department there is no employees (A newly created department)

```
SELECT DEPT.DEPTNAME, EMP.EMPNAME
FROM DEPT
LEFT JOIN EMP ON DEPT.DEPTNO = EMP.DEPTNO;
```

Question 17

List all the employees name and their department name even if for an employee there is no department assigned (A newly joined employee).

```
SELECT EMP.EMPNAME, DEPT.DEPTNAME
FROM EMP
LEFT JOIN DEPT ON EMP.DEPTNO = DEPT.DEPTNO;
```

Question 18

List all the departments name and their employees name irrespective whether for a department any employees are there are not, and irrespective whether for an employee there is any department assigned or not.

```
SELECT DEPT.DEPTNAME, EMP.EMPNAME
FROM DEPT
FULL OUTER JOIN EMP ON DEPT.DEPTNO = EMP.DEPTNO;
```

DBMS Assignment 7

By Himanshu Sardana

Question 1

Create the above 3 tables and insert those records

```
CREATE TABLE SalesPeople (  
    Snum INTEGER PRIMARY KEY,  
    Sname VARCHAR(30),  
    City VARCHAR(30),  
    Comm FLOAT  
);  
  
CREATE TABLE Customers (  
    Cnum INTEGER PRIMARY KEY,  
    Cname VARCHAR(30),  
    City VARCHAR(30) NOT NULL,  
    Snum INTEGER,  
    FOREIGN KEY (Snum) REFERENCES SalesPeople(Snum)  
);  
  
CREATE TABLE Orders (  
    Onum INTEGER PRIMARY KEY,  
    Amt FLOAT,  
    Odate DATE,  
    Cnum INTEGER,  
    Snum INTEGER,  
    FOREIGN KEY (Cnum) REFERENCES Customers(Cnum),  
    FOREIGN KEY (Snum) REFERENCES SalesPeople(Snum)  
);  
  
INSERT INTO SalesPeople VALUES (1001, 'Peel', 'London', 0.12);  
INSERT INTO SalesPeople VALUES (1002, 'Serres', 'Sanjose', 0.13);  
INSERT INTO SalesPeople VALUES (1004, 'Motika', 'London', 0.11);  
INSERT INTO SalesPeople VALUES (1007, 'Rifkin', 'Barcelona', 0.15);  
INSERT INTO SalesPeople VALUES (1003, 'Axel', 'New York', 0.10);  
  
INSERT INTO Customers VALUES (2001, 'Hoffman', 'London', 1001);  
INSERT INTO Customers VALUES (2002, 'Giovanni', 'Rome', 1003);  
INSERT INTO Customers VALUES (2003, 'Liu', 'Sanjose', 1002);  
INSERT INTO Customers VALUES (2004, 'Grass', 'Berlin', 1002);  
INSERT INTO Customers VALUES (2006, 'Clemens', 'London', 1001);  
INSERT INTO Customers VALUES (2008, 'Cisneros', 'Sanjose', 1003);  
INSERT INTO Customers VALUES (2007, 'Pereira', 'Rome', 1003);  
  
INSERT INTO Orders VALUES (3001, 18.69, '3-10-1990', 2008, 1002);  
INSERT INTO Orders VALUES (3003, 767.19, '3-10-1990', 2001, 1001);  
INSERT INTO Orders VALUES (3002, 1900.10, '3-10-1990', 2007, 1004);
```

```
INSERT INTO Orders VALUES (3005, 5160.45, '3-10-1990', 2003, 1002);
INSERT INTO Orders VALUES (3006, 1098.16, '3-10-1990', 2002, 1007);
INSERT INTO Orders VALUES (3009, 1713.23, '4-10-1990', 2004, 1002);
INSERT INTO Orders VALUES (3007, 75.75, '4-10-1990', 2004, 1001);
INSERT INTO Orders VALUES (3008, 4723.00, '4-10-1990', 2006, 1001);
INSERT INTO Orders VALUES (3010, 1309.95, '4-10-1990', 2004, 1002);
INSERT INTO Orders VALUES (3011, 9891.88, '4-10-1990', 2006, 1001);
```

Question 2

Display all the salesperson whose all orders worth is more than Rs. 2000.

```
SELECT Sname FROM SalesPeople WHERE Snum IN (
    SELECT Snum FROM Orders GROUP BY Snum HAVING SUM(Amt) > 2000
);
```

Question 3

Display the salesperson id handling at least two customers.

```
SELECT Snum FROM Customers GROUP BY Snum HAVING COUNT(Cnum) >= 2;
```

Question 4

Display the salesperson names handling at least two customers.

```
SELECT Sname FROM SalesPeople WHERE Snum IN (
    SELECT Snum FROM Customers GROUP BY Snum HAVING COUNT(Cnum) >= 2
);
```

Question 5

Display the number of Salespeople belonging to London and Paris.

```
SELECT City, COUNT(Snum) FROM SalesPeople WHERE City IN ('London', 'Paris') GROUP BY City;
```

Question 6

Display the customer names corresponding to salesperson belonging to London and Paris.

```
SELECT Cname FROM Customers WHERE Snum IN (
    SELECT Snum FROM SalesPeople WHERE City IN ('London', 'Paris')
);
```

Question 7

Display the customer names whose order amount is worth 1200

```
SELECT Cname FROM Customers WHERE Cnum IN (  
    SELECT Cnum FROM Orders WHERE Amt = 1200  
);
```

Question 8

Display the customer names who belong to the same city as their salesperson.

```
SELECT Cname FROM Customers WHERE City IN (  
    SELECT City FROM SalesPeople WHERE Snum = Customers.Snum  
);
```

Question 9

Display the salesperson names having commission more than 10 percent and who handled more than two customers.

```
SELECT Sname FROM SalesPeople WHERE Comm > 0.10 AND Snum IN (  
    SELECT Snum FROM Customers GROUP BY Snum HAVING COUNT(Cnum) > 2  
);
```

Question 10

Display the customer's name who orders exactly one order and his salesperson name

```
SELECT Cname, Sname FROM Customers, SalesPeople WHERE Cnum IN (  
    SELECT Cnum FROM Orders GROUP BY Cnum HAVING COUNT(Onum) = 1  
) AND Customers.Snum = SalesPeople.Snum;
```

Question 11

Insert the details of a salesperson in the table who is a new joined and did not handle any customer. Write a query for the salesperson who has not handle any customer.

```
INSERT INTO SalesPeople VALUES (1008, 'New Salesperson', 'New City', 0.10);
```

Question 12

Write a query that counts the number of Salespeople registering orders for each day.

```
SELECT Odate, COUNT(Snum) FROM Orders GROUP BY Odate;
```

Question 13

Find out the no. of Salesperson who belongs to same city and have same commission percentage

```
SELECT City, Comm, COUNT(Snum) FROM SalesPeople GROUP BY City, Comm HAVING COUNT(Snum) > 1;
```


DBMS Assignment 8

By Himanshu Sardana

Creating Tables

```
CREATE TABLE Sailors (  
    sid INTEGER PRIMARY KEY,  
    sname VARCHAR(50),  
    rating INTEGER,  
    age REAL  
);  
  
CREATE TABLE Boats (  
    bid INTEGER PRIMARY KEY,  
    bname VARCHAR(50),  
    color VARCHAR(20)  
);  
  
CREATE TABLE Reserves (  
    sid INTEGER,  
    bid INTEGER,  
    day DATE,  
    PRIMARY KEY (sid, bid, day),  
    FOREIGN KEY (sid) REFERENCES Sailors(sid),  
    FOREIGN KEY (bid) REFERENCES Boats(bid)  
);
```

Inserting Data

```
INSERT INTO Sailors (sid, sname, rating, age) VALUES (22, 'Dustin', 7, 45.0);  
INSERT INTO Sailors (sid, sname, rating, age) VALUES (29, 'Brutus', 1, 33.0);  
INSERT INTO Sailors (sid, sname, rating, age) VALUES (31, 'Lubber', 8, 55.5);  
INSERT INTO Sailors (sid, sname, rating, age) VALUES (32, 'Andy', 8, 25.5);  
INSERT INTO Sailors (sid, sname, rating, age) VALUES (58, 'Rusty', 10, 35.0);  
INSERT INTO Sailors (sid, sname, rating, age) VALUES (64, 'Horatio', 7, 35.0);  
INSERT INTO Sailors (sid, sname, rating, age) VALUES (71, 'Zorba', 10, 16.0);  
INSERT INTO Sailors (sid, sname, rating, age) VALUES (74, 'Horatio', 9, 35.0);  
INSERT INTO Sailors (sid, sname, rating, age) VALUES (85, 'Art', 3, 25.5);  
INSERT INTO Sailors (sid, sname, rating, age) VALUES (95, 'Bob', 3, 63.5);  
  
INSERT INTO Boats (bid, bname, color) VALUES (101, 'Interlake', 'blue');  
INSERT INTO Boats (bid, bname, color) VALUES (102, 'Interlake', 'red');  
INSERT INTO Boats (bid, bname, color) VALUES (103, 'Clipper', 'green');  
INSERT INTO Boats (bid, bname, color) VALUES (104, 'Marine', 'red');  
  
INSERT INTO Reserves (sid, bid, day) VALUES (22, 101, '1998-10-10');  
INSERT INTO Reserves (sid, bid, day) VALUES (22, 102, '1998-10-10');  
INSERT INTO Reserves (sid, bid, day) VALUES (22, 103, '1998-10-08');  
INSERT INTO Reserves (sid, bid, day) VALUES (31, 102, '1998-10-07');  
INSERT INTO Reserves (sid, bid, day) VALUES (31, 103, '1998-11-10');
```

```
INSERT INTO Reserves (sid, bid, day) VALUES (31, 104, '1998-11-06');
INSERT INTO Reserves (sid, bid, day) VALUES (64, 101, '1998-11-12');
INSERT INTO Reserves (sid, bid, day) VALUES (64, 104, '1998-09-05');
INSERT INTO Reserves (sid, bid, day) VALUES (74, 103, '1998-09-08');
```

Question 1

Find the names of sailors who have reserved a red or a green boat.

```
SELECT sid FROM Reserves
WHERE bid IN (
    SELECT bid FROM Boats
    WHERE color = 'red' OR color = 'green'
);
```

Question 2

Find the names of sailors who have reserved both a red and a green boat

```
SELECT sid FROM Reserves
WHERE bid IN (
    SELECT bid FROM Boats
    WHERE color = 'red'
)
AND sid IN (
    SELECT sid FROM Reserves
    WHERE bid IN (
        SELECT bid FROM Boats
        WHERE color = 'green'
    )
);
```

Question 3

Find the names of sailors who have reserved boat 103

```
SELECT sname FROM Sailors
WHERE sid IN (
    SELECT sid FROM Reserves
    WHERE bid = 103
);
```

Question 4

Find the names of sailors who have reserved a red boat.

```

SELECT sname FROM Sailors
  WHERE sid IN (
    SELECT sid FROM Reserves
    WHERE bid IN (
      SELECT bid FROM Boats
      WHERE color = 'red'
    )
  );

```

Question 5

Find the names of sailors who have NOT reserved a red boat.

```

SELECT sname FROM Sailors
  WHERE sid NOT IN (
    SELECT sid FROM Reserves
    WHERE bid IN (
      SELECT bid FROM Boats
      WHERE color = 'red'
    )
  );

```

Question 6

Find the names of sailors who have reserved at least one boat.

```

SELECT sname FROM Sailors
  WHERE sid IN (
    SELECT sid FROM Reserves
  );

```

Question 7

Find sailors whose rating is better than some sailor called Horatio

```

SELECT sname FROM Sailors
  WHERE rating > (
    SELECT rating FROM Sailors
    WHERE sname = 'Horatio'
  );

```

Question 8

Find sailors whose rating is better than every sailor called Horatio

```
SELECT sname FROM Sailors
WHERE rating > ALL (
    SELECT rating FROM Sailors
    WHERE sname = 'Horatio'
);
```

Question 9

Find the sailors with the highest rating.

```
SELECT sname FROM Sailors
WHERE rating = (
    SELECT MAX(rating) FROM Sailors
);
```

Question 10

Find the names of sailors who have reserved all boats.

```
SELECT sname FROM Sailors
WHERE NOT EXISTS (
    SELECT bid FROM Boats
    WHERE NOT EXISTS (
        SELECT bid FROM Reserves
        WHERE Reserves.sid = Sailors.sid
        AND Reserves.bid = Boats.bid
    )
);
```

DBMS Assignment 9

By Himanshu Sardana

Question 1

WAP to find the greatest of three numbers

```
DECLARE
  a number := 1;
  b number := 2;
  c number := 3;
BEGIN
  IF a > b THEN
    IF a > c THEN
      dbms_output.put_line(a);
    ELSE
      dbms_output.put_line(c);
    END IF;
  ELSE
    IF b > c THEN
      dbms_output.put_line(b);
    ELSE
      dbms_output.put_line(c);
    END IF;
  END IF;
END;
```

Question 2

WAP to check whether number is odd or even

```
DECLARE
  a number := 7;
BEGIN
  IF MOD(a, 2) = 0 THEN
    dbms_output.put_line('Even');
  ELSE
    dbms_output.put_line('Odd');
  END IF;
END;
```

Question 3

WAP to find the grade. Consider the following:

Marks > 80 A grade

Marks > 70 B grade

Marks > 50 C grade

Marks > 40 D grade

Marks < 40 E grade

```
DECLARE
    marks number := 75;
BEGIN
    IF marks > 80 THEN
        dbms_output.put_line('A');
    ELSIF marks > 70 THEN
        dbms_output.put_line('B');
    ELSIF marks > 50 THEN
        dbms_output.put_line('C');
    ELSIF marks > 40 THEN
        dbms_output.put_line('D');
    ELSE
        dbms_output.put_line('E');
    END IF;
END;
```

Question 4

WAP to print the table of a given number (use for loop)

```
DECLARE
    a number := 5;
BEGIN
    for i in 1..10 loop
        dbms_output.put_line(a * i);
    end loop;
END;
```

Question 5

WAP to find the factorial of a given number (use while loop)

```
DECLARE
    a number := 5;
    fact number := 1;
BEGIN
    WHILE a > 0 LOOP
        fact := fact * a;
        a := a - 1;
    END LOOP;
    dbms_output.put_line(fact);
END;
```

Question 6

WAP to find the fibonacci series

```
DECLARE
  a number := 0;
  b number := 1;
  c number;
BEGIN
  for i in 1..10 loop
    dbms_output.put_line(a);
    c := a + b;
    a := b;
    b := c;
  end loop;
END;
```

Question 7

WAP to find the reverse of a number

```
DECLARE
  a number := 12345;
  rev number := 0;
BEGIN
  WHILE a > 0 LOOP
    rev := rev * 10 + MOD(a, 10);
    a := TRUNC(a / 10);
  END LOOP;
  dbms_output.put_line(rev);
END;
```

Question 8

Write PL/SQL block that performs addition (+), subtraction (-), multiplication (*) and division (/) of two numbers as choice by the user

```
DECLARE
  a number := 10;
  b number := 5;
  choice number := 1; -- 1 for addition, 2 for subtraction, 3 for multiplication, 4 for
division
BEGIN
  CASE choice
    WHEN 1 THEN
      dbms_output.put_line(a + b);
    WHEN 2 THEN
      dbms_output.put_line(a - b);
    WHEN 3 THEN
      dbms_output.put_line(a * b);
    WHEN 4 THEN
```

```
dbms_output.put_line(a / b);  
ELSE  
dbms_output.put_line('Invalid choice');  
END CASE;  
END;
```

Question 9

Write PL/SQL block to print 5, 10, 15,20 by using For Loop

```
BEGIN  
FOR i IN 1..5 LOOP  
dbms_output.put_line(i * 5);  
END LOOP;  
END;
```

Question 10

Write PL/SQL block to display welcome message like good morning, good afternoon, good night depending on system time.

```
DECLARE  
current_hour number;  
BEGIN  
SELECT TO_CHAR(SYSDATE, 'HH24') INTO current_hour FROM DUAL;  
  
IF current_hour < 12 THEN  
dbms_output.put_line('Good Morning');  
ELSIF current_hour < 18 THEN  
dbms_output.put_line('Good Afternoon');  
ELSE  
dbms_output.put_line('Good Night');  
END IF;  
END;
```


DBMS Assignment 10

By Himanshu Sardana

Question 1

Create a table EMP (eno, ename, bp, da, hra, total). Initially, the 'total' column values for all the employees are null. Write a stored procedure that accepts the eno of an employee as an input and computes his total by adding bp + da + hra. Your pl/sql code should send the total to the main block for printing and update it on the emp table.

```
CREATE TABLE EMP (  
    eno NUMBER PRIMARY KEY,  
    ename VARCHAR(50),  
    bp NUMBER,  
    da NUMBER,  
    hra NUMBER,  
    total NUMBER  
);  
  
CREATE OR REPLACE PROCEDURE calculate_total (p_eno IN NUMBER) IS  
    v_bp EMP.bp%TYPE;  
    v_da EMP.da%TYPE;  
    v_hra EMP.hra%TYPE;  
    v_total EMP.total%TYPE;  
BEGIN  
    -- Fetch the employee details  
    SELECT bp, da, hra INTO v_bp, v_da, v_hra FROM EMP WHERE eno = p_eno;  
  
    -- Calculate the total  
    v_total := v_bp + v_da + v_hra;  
  
    -- Update the total in the table  
    UPDATE EMP SET total = v_total WHERE eno = p_eno;  
  
    -- Print the total  
    DBMS_OUTPUT.PUT_LINE('Total for employee ' || p_eno || ': ' || v_total);  
END;  
/
```

Question 2

Write a local function that will accept a number as an input, compute it factorial, and send it to the main block for printing

```
CREATE OR REPLACE FUNCTION factorial (n IN NUMBER) RETURN NUMBER IS  
    v_result NUMBER := 1;  
BEGIN  
    FOR i IN 1..n LOOP  
        v_result := v_result * i;  
    END LOOP;  
    RETURN v_result;  
END;
```

```

        END LOOP;
        RETURN v_result;
    END;
/

DECLARE
    v_number NUMBER := 5; -- Example number
    v_factorial NUMBER;
BEGIN
    v_factorial := factorial(v_number);
    DBMS_OUTPUT.PUT_LINE('Factorial of ' || v_number || ' is: ' || v_factorial);
END;

```

Question 3

Write a stored procedure that will accept a number, check whether positive, negative or zero and accordingly print the message.

```

CREATE OR REPLACE PROCEDURE check_number (p_number IN NUMBER) IS
BEGIN
    IF p_number > 0 THEN
        DBMS_OUTPUT.PUT_LINE('The number ' || p_number || ' is positive.');
```

```

    ELSIF p_number < 0 THEN
        DBMS_OUTPUT.PUT_LINE('The number ' || p_number || ' is negative.');
```

```

    ELSE
        DBMS_OUTPUT.PUT_LINE('The number is zero.');
```

```

    END IF;
END;
/

DECLARE
    v_number NUMBER := -10;
BEGIN
    check_number(v_number);
END;

```

Question 4

Write a stored function that will find the sum of elements from 1 to n and send the result to the main block. 'n' should send as the input to the called function.

```

CREATE OR REPLACE FUNCTION sum_of_elements (n IN NUMBER) RETURN NUMBER IS
    v_sum NUMBER := 0;
BEGIN
    FOR i IN 1..n LOOP
        v_sum := v_sum + i;
    END LOOP;
    RETURN v_sum;
END;
/

```

```

DECLARE
    v_n NUMBER := 10; -- Example number
    v_sum NUMBER;
BEGIN
    v_sum := sum_of_elements(v_n);
    DBMS_OUTPUT.PUT_LINE('Sum of elements from 1 to ' || v_n || ' is: ' || v_sum);
END;

```

Question 5

Write a local procedure that will accept two ranges n1 and n2, find the sum of the average of elements between these ranges and return the result to the main block

```

CREATE OR REPLACE PROCEDURE sum_of_average (n1 IN NUMBER, n2 IN NUMBER, p_result OUT NUMBER)
IS
    v_sum NUMBER := 0;
    v_count NUMBER := 0;
BEGIN
    FOR i IN n1..n2 LOOP
        v_sum := v_sum + i;
        v_count := v_count + 1;
    END LOOP;

    IF v_count > 0 THEN
        p_result := v_sum / v_count;
    ELSE
        p_result := 0;
    END IF;
END;
/

DECLARE
    v_n1 NUMBER := 1; -- Example range start
    v_n2 NUMBER := 10; -- Example range end
    v_result NUMBER;
BEGIN
    sum_of_average(v_n1, v_n2, v_result);
    DBMS_OUTPUT.PUT_LINE('Average of elements between ' || v_n1 || ' and ' || v_n2 || ' is: ' || v_result);
END;

```

Question 6

Write a recursive function to display 'hello world' 10 times on the screen. The function should accept the string as input from the main block

```

CREATE OR REPLACE FUNCTION display_hello (p_count IN NUMBER, p_message IN VARCHAR2) RETURN
NUMBER IS
BEGIN
    IF p_count > 0 THEN
        DBMS_OUTPUT.PUT_LINE(p_message);
    END IF;
    RETURN display_hello(p_count - 1, p_message);
END;

```

```

        RETURN display_hello(p_count - 1, p_message);
    ELSE
        RETURN 0;
    END IF;
END;
/

DECLARE
    v_count NUMBER := 10; -- Number of times to display
    v_message VARCHAR2(50) := 'Hello World'; -- Message to display
BEGIN
    display_hello(v_count, v_message);
END;

```

Question 7

Consider a table student (rollno number, name varchar (30), sub-id number, mark1 number, mark2 number, mark3 number). Q7-Q10 are based on the student table.

Write a stored procedure 'p1', which will accept the rollno of a student as the input, and then find the sum of the three subject marks and store it in a variable 'total'. Send this 'total' to the main block of the program and display it.

```

CREATE TABLE student (
    rollno NUMBER PRIMARY KEY,
    name VARCHAR2(30),
    sub_id NUMBER,
    mark1 NUMBER,
    mark2 NUMBER,
    mark3 NUMBER
);

CREATE OR REPLACE PROCEDURE p1 (p_rollno IN NUMBER, p_total OUT NUMBER) IS
    v_mark1 student.mark1%TYPE;
    v_mark2 student.mark2%TYPE;
    v_mark3 student.mark3%TYPE;
BEGIN
    SELECT mark1, mark2, mark3 INTO v_mark1, v_mark2, v_mark3 FROM student WHERE rollno =
p_rollno;

    p_total := v_mark1 + v_mark2 + v_mark3;
END;
/

DECLARE
    v_rollno NUMBER := 1;
    v_total NUMBER;
BEGIN
    p1(v_rollno, v_total);
    DBMS_OUTPUT.PUT_LINE('Total marks for roll number ' || v_rollno || ': ' || v_total);
END;

```

Question 8

Write a local function 'fun2', which will call the procedure 'p1' of Q7 by passing the rollno of a student. It then finds the average of the total marks and stores it in a variable 'avg'. Send this 'avg' to the main block of the program and display it.

```
CREATE OR REPLACE FUNCTION fun2 (p_rollno IN NUMBER) RETURN NUMBER IS
    v_total NUMBER;
    v_avg NUMBER;
BEGIN
    p1(p_rollno, v_total);
    v_avg := v_total / 3; -- Assuming 3 subjects
    RETURN v_avg;
END;
/

DECLARE
    v_rollno NUMBER := 1;
    v_avg NUMBER;
BEGIN
    v_avg := fun2(v_rollno);
    DBMS_OUTPUT.PUT_LINE('Average marks for roll number ' || v_rollno || ': ' || v_avg);
END;
```

Question 9

Write a stored function 'fun3' which will accept the rollno of a student as the input and return the highest marks among the three subjects to the main block.

```
CREATE OR REPLACE FUNCTION fun3 (p_rollno IN NUMBER) RETURN NUMBER IS
    v_mark1 student.mark1%TYPE;
    v_mark2 student.mark2%TYPE;
    v_mark3 student.mark3%TYPE;
    v_highest NUMBER;
BEGIN
    SELECT mark1, mark2, mark3 INTO v_mark1, v_mark2, v_mark3 FROM student WHERE rollno =
p_rollno;

    v_highest := GREATEST(v_mark1, v_mark2, v_mark3);
    RETURN v_highest;
END;
/

DECLARE
    v_rollno NUMBER := 1;
    v_highest NUMBER;
BEGIN
    v_highest := fun3(v_rollno);
    DBMS_OUTPUT.PUT_LINE('Highest marks for roll number ' || v_rollno || ': ' || v_highest);
END;
```

Question 10

Write a local procedure p2, which will call the procedure 'p1' of Q7 by passing the rollno of a student. It then received the total marks from the procedure 'p1' and sent the student's three subject marks and the total marks to the main block for printing

```
CREATE OR REPLACE PROCEDURE p2 (p_rollno IN NUMBER) IS
    v_mark1 student.mark1%TYPE;
    v_mark2 student.mark2%TYPE;
    v_mark3 student.mark3%TYPE;
    v_total NUMBER;
BEGIN
    SELECT mark1, mark2, mark3 INTO v_mark1, v_mark2, v_mark3 FROM student WHERE rollno =
p_rollno;

    p1(p_rollno, v_total);

    DBMS_OUTPUT.PUT_LINE('Marks for roll number ' || p_rollno || ': ' || v_mark1 || ', ' ||
v_mark2 || ', ' || v_mark3);
    DBMS_OUTPUT.PUT_LINE('Total marks: ' || v_total);
END;
/

DECLARE
    v_rollno NUMBER := 1;
BEGIN
    p2(v_rollno);
END;
```

DBMS Assignment 11

By Himanshu Sardana

```
CREATE TABLE Student (  
    Rollno NUMBER PRIMARY KEY,  
    name VARCHAR(30),  
    age NUMBER,  
    mark1 NUMBER,  
    mark2 NUMBER,  
    mark3 NUMBER,  
    total NUMBER  
);
```

Question 1

Write a pl/sql code using cursor, which will delete all those records from the Student table where age < 25 and insert those records into another table called Student-minor

```
CREATE TABLE Student_minor (  
    Rollno NUMBER PRIMARY KEY,  
    name VARCHAR(30),  
    age NUMBER,  
    mark1 NUMBER,  
    mark2 NUMBER,  
    mark3 NUMBER,  
    total NUMBER  
);  
  
DECLARE  
    CURSOR c1 IS SELECT * FROM Student WHERE age < 25;  
    v_student Student%ROWTYPE;  
BEGIN  
    FOR v_student IN c1 LOOP  
        INSERT INTO Student_minor VALUES v_student;  
        DELETE FROM Student WHERE Rollno = v_student.Rollno;  
    END LOOP;  
END;  
/
```

Question 2

Write a pl/sql code using cursor, which will find the total marks of each student and update the total column (assume that initially, the total is zero for all the students).

```

DECLARE
    CURSOR c1 IS SELECT Rollno, mark1, mark2, mark3 FROM Student;
    v_total NUMBER;
BEGIN
    FOR v_student IN c1 LOOP
        v_total := v_student.mark1 + v_student.mark2 + v_student.mark3;
        UPDATE Student SET total = v_total WHERE Rollno = v_student.Rollno;
    END LOOP;
END;
/

```

Question 3

Write a pl/sql code using cursor to find out how many students are there whose total marks are greater than 90, and then display their details.

```

DECLARE
    CURSOR c1 IS SELECT * FROM Student WHERE total > 90;
    v_student Student%ROWTYPE;
BEGIN
    FOR v_student IN c1 LOOP
        DBMS_OUTPUT.PUT_LINE('Rollno: ' || v_student.Rollno || ', Name: ' || v_student.name
|| ', Total Marks: ' || v_student.total);
    END LOOP;
END;
/

```

Question 4

Write a pl/sql code using cursor to find the highest and lowest marks and display the corresponding student's details

```

DECLARE
    CURSOR c1 IS SELECT * FROM Student;
    v_highest Student%ROWTYPE;
    v_lowest Student%ROWTYPE;
    v_highest_total NUMBER := 0;
    v_lowest_total NUMBER := 100;
BEGIN
    FOR v_student IN c1 LOOP
        IF v_student.total > v_highest_total THEN
            v_highest_total := v_student.total;
            v_highest := v_student;
        END IF;
        IF v_student.total < v_lowest_total THEN
            v_lowest_total := v_student.total;
            v_lowest := v_student;
        END IF;
    END LOOP;
END;

```



```

        DBMS_OUTPUT.PUT_LINE('Highest Marks: Rollno: ' || v_highest.Rollno || ', Name: ' ||
v_highest.name || ', Total Marks: ' || v_highest.total);
        DBMS_OUTPUT.PUT_LINE('Lowest Marks: Rollno: ' || v_lowest.Rollno || ', Name: ' ||
v_lowest.name || ', Total Marks: ' || v_lowest.total);
    END;
/

```

Question 5

Write a pl/sql code using cursor to find the average mark of all the students and display it on the screen.

```

DECLARE
    CURSOR c1 IS SELECT AVG(total) AS avg_total FROM Student;
    v_avg_total NUMBER;
BEGIN
    FOR v_student IN c1 LOOP
        v_avg_total := v_student.avg_total;
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('Average Marks: ' || v_avg_total);
END;
/

```

Question 6

Write a stored procedure using a parameterized cursor, which will display the student details whose rollno is passing as a parameter to the cursor from the stored procedure

```

CREATE OR REPLACE PROCEDURE display_student(p_rollno IN NUMBER) IS
    CURSOR c1 IS SELECT * FROM Student WHERE Rollno = p_rollno;
    v_student Student%ROWTYPE;
BEGIN
    OPEN c1;
    LOOP
        FETCH c1 INTO v_student;
        EXIT WHEN c1%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE('Rollno: ' || v_student.Rollno || ', Name: ' || v_student.name
|| ', Total Marks: ' || v_student.total);
    END LOOP;
END display_student;
/

```

```

CREATE TABLE EMP (
    empno NUMBER PRIMARY KEY,
    ename VARCHAR(30),
    job VARCHAR(30),
    sal NUMBER,
    deptno NUMBER
);

```

Question 7

Write a PL/SQL code to demonstrate %TYPE and %ROWTYPE to display details of employees in EMP table.

```
DECLARE
    v_empno EMP.empno%TYPE;
    v_ename EMP.ename%TYPE;
    v_job EMP.job%TYPE;
    v_sal EMP.sal%TYPE;
    v_deptno EMP.deptno%TYPE;
    v_emp_record EMP%ROWTYPE;
BEGIN
    SELECT empno, ename, job, sal, deptno INTO v_emp_record FROM EMP WHERE empno = 1;
    v_empno := v_emp_record.empno;
    v_ename := v_emp_record.ename;
    v_job := v_emp_record.job;
    v_sal := v_emp_record.sal;
    v_deptno := v_emp_record.deptno;

    DBMS_OUTPUT.PUT_LINE('Emp No: ' || v_empno || ', Name: ' || v_ename || ', Job: ' ||
v_job || ', Salary: ' || v_sal || ', Dept No: ' || v_deptno);
END;
/
```

Question 8

Write a stored function to display the empno, ename, and job of employees of a department for EMP table using a parameterized cursor where deptno will be sent as a parameter to the cursor from the stored function

```
CREATE OR REPLACE FUNCTION display_emp_by_dept(p_deptno IN NUMBER) RETURN SYS_REFCURSOR IS
    v_cursor SYS_REFCURSOR;
BEGIN
    OPEN v_cursor FOR SELECT empno, ename, job FROM EMP WHERE deptno = p_deptno;
    RETURN v_cursor;
END display_emp_by_dept;
/
```

Question 9

Write a local function to display the employee number and name of the top 'n' highest-paid Employees using parameterized cursor. The value of 'n' is passed to the cursor as a parameter from the local function

```
CREATE OR REPLACE FUNCTION display_top_n_employees(p_n IN NUMBER) RETURN SYS_REFCURSOR IS
    v_cursor SYS_REFCURSOR;
BEGIN
    OPEN v_cursor FOR SELECT empno, ename FROM EMP ORDER BY sal DESC FETCH FIRST p_n ROWS
ONLY;
    RETURN v_cursor;
END display_top_n_employees;
/
```

Question 10

Write a local procedure to calculate the total salary of the first 'n' records of EMP table using parameterized cursor. The value of 'n' is passed to cursor as a parameter from the local procedure.

```
CREATE OR REPLACE PROCEDURE calculate_total_salary(p_n IN NUMBER) IS
    cursor v_cursor IS SELECT sal FROM EMP;
    v_total_salary NUMBER := 0;
    v_count NUMBER := 0;
BEGIN
    FOR v_emp IN v_cursor LOOP
        IF v_count < p_n THEN
            v_total_salary := v_total_salary + v_emp.sal;
            v_count := v_count + 1;
        ELSE
            EXIT;
        END IF;
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('Total Salary of first ' || p_n || ' records: ' || v_total_salary);
END calculate_total_salary;
/
```

Question 11

Write a PL/SQL program to demonstrate the following exceptions:

- When Too Many Rows
- When No Data Found
- When Others

```
DECLARE
    v_empno EMP.empno%TYPE;
    v_ename EMP.ename%TYPE;
    v_job EMP.job%TYPE;
    v_sal EMP.sal%TYPE;
    v_deptno EMP.deptno%TYPE;
BEGIN
    SELECT empno, ename, job, sal, deptno INTO v_empno, v_ename, v_job, v_sal, v_deptno FROM
EMP WHERE empno = 1;
    DBMS_OUTPUT.PUT_LINE('Emp No: ' || v_empno || ', Name: ' || v_ename || ', Job: ' ||
v_job || ', Salary: ' || v_sal || ', Dept No: ' || v_deptno);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No data found for the given empno.');
```

```
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('Too many rows returned.');
```

```
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' || SQLERRM);
END;
/
```

Question 12

Write a PL/SQL program to demonstrate the User Defined Exceptions.

```
DECLARE
    v_empno EMP.empno%TYPE;
    v_ename EMP.ename%TYPE;
    v_job EMP.job%TYPE;
    v_sal EMP.sal%TYPE;
    v_deptno EMP.deptno%TYPE;
    emp_not_found EXCEPTION;
BEGIN
    SELECT empno, ename, job, sal, deptno INTO v_empno, v_ename, v_job, v_sal, v_deptno FROM
EMP WHERE empno = 1;
    DBMS_OUTPUT.PUT_LINE('Emp No: ' || v_empno || ', Name: ' || v_ename || ', Job: ' ||
v_job || ', Salary: ' || v_sal || ', Dept No: ' || v_deptno);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE emp_not_found;
    WHEN emp_not_found THEN
        DBMS_OUTPUT.PUT_LINE('Employee not found.');
```

```
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' || SQLERRM);
END;
/
```

DBMS Assignment 12

By Himanshu Sardana

EMP (ENO, ENAME, JOB, AGE, SALARY, COMMISSION, DEPTNO).

DEPT (DEPTNO, DNAME)

(Deptno of EMP is a foreign key which refers to the Deptno of Dept Table)

```
CREATE TABLE DEPT (  
    DEPTNO NUMBER(3),  
    DNAME VARCHAR(255)  
);  
  
CREATE TABLE EMP (  
    ENO NUMBER(10),  
    ENAME VARCHAR(255),  
    JOB VARCHAR(25),  
    AGE NUMBER(3),  
    SALARY NUMBER(7),  
    COMMISSION NUMBER(7),  
    DEPTNO NUMBER(3),  
    CONSTRAINT FK_DEPTNO FOREIGN KEY (DEPTNO) REFERENCES DEPT (DEPTNO)  
);
```

Question 1

Write a database trigger on the EMP table, which will store each deleted record from the EMP table in another table called EMP_OLD

```
CREATE TABLE EMP_OLD (  
    ENO NUMBER(10),  
    ENAME VARCHAR(255),  
    JOB VARCHAR(25),  
    AGE NUMBER(3),  
    SALARY NUMBER(7),  
    COMMISSION NUMBER(7),  
    DEPTNO NUMBER(3)  
);  
  
CREATE OR REPLACE TRIGGER trg_emp_delete  
BEFORE DELETE ON EMP  
FOR EACH ROW  
BEGIN  
    INSERT INTO EMP_OLD (ENO, ENAME, JOB, AGE, SALARY, COMMISSION, DEPTNO)  
    VALUES (:OLD.ENO, :OLD.ENAME, :OLD.JOB, :OLD.AGE, :OLD.SALARY, :OLD.COMMISSION,  
:OLD.DEPTNO);  
END;  
/
```

Question 2

Create a database trigger on the EMP table, verifying that no employee is under 25 while entering the details (ENO, AGE) into the EMP table

```
CREATE OR REPLACE TRIGGER trg_emp_age
BEFORE INSERT OR UPDATE ON EMP
FOR EACH ROW
BEGIN
    IF :NEW.AGE < 25 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Employee age must be 25 or older');
    END IF;
END;
/
```

Question 3

Write a database trigger on the EMP table, which verifies that each employee's updated salary should not less than their old salary.

```
CREATE OR REPLACE TRIGGER trg_emp_salary
BEFORE UPDATE OF SALARY ON EMP
FOR EACH ROW
BEGIN
    IF :NEW.SALARY < :OLD.SALARY THEN
        RAISE_APPLICATION_ERROR(-20002, 'New salary must be greater than or equal to old salary');
    END IF;
END;
/
```

Question 4

Write a database trigger which is an example of a statement-level trigger.

```
CREATE OR REPLACE TRIGGER trg_emp_statement
BEFORE INSERT ON EMP
BEGIN
    DBMS_OUTPUT.PUT_LINE('A new employee is being added to the EMP table');
END;
/
```

Question 5

Create a trigger on the EMP table to convert supplied values of the ENAME column to uppercase for INSERT and UPDATE statements

```

CREATE OR REPLACE TRIGGER trg_emp_uppercase
BEFORE INSERT OR UPDATE OF ENAME ON EMP
FOR EACH ROW
BEGIN
    :NEW.ENAME := UPPER (:NEW.ENAME);
END;
/

```

Question 6

Create a trigger on the emp table, which shows the old and new values of ENAME after every updation on the ENAME of the EMP table.

```

CREATE OR REPLACE TRIGGER trg_emp_ename_update
AFTER UPDATE OF ENAME ON EMP
FOR EACH ROW
BEGIN
    DBMS_OUTPUT.PUT_LINE('Old ENAME: ' || :OLD.ENAME || ', New ENAME: ' || :NEW.ENAME);
END;
/

```

Question 7

Write a trigger to ensure that an employee's commission cannot be greater than his salary

```

CREATE OR REPLACE TRIGGER trg_emp_commission
BEFORE INSERT OR UPDATE OF COMMISSION ON EMP
FOR EACH ROW
BEGIN
    IF :NEW.COMMISSION > :NEW.SALARY THEN
        RAISE_APPLICATION_ERROR(-20003, 'Commission cannot be greater than salary');
    END IF;
END;
/

```

Question 8

Create a trigger so that no operation can be performed on the EMP table on Sunday

```

CREATE OR REPLACE TRIGGER trg_emp_no_sunday
BEFORE INSERT OR UPDATE OR DELETE ON EMP
DECLARE
    v_day VARCHAR2(10);
BEGIN
    SELECT TO_CHAR(SYSDATE, 'Day') INTO v_day FROM DUAL;
    IF TRIM(v_day) = 'Sunday' THEN
        DBMS_OUTPUT.PUT_LINE('No operations can be performed on Sunday');
    END IF;
END;
/

```

Question 9

Create a trigger to implement the primary key constraint on column ENO of table EMP.

```
CREATE OR REPLACE TRIGGER trg_emp_primary_key
BEFORE INSERT ON EMP
FOR EACH ROW
DECLARE
    v_count NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_count FROM EMP WHERE ENO = :NEW.ENO;
    IF v_count > 0 THEN
        RAISE_APPLICATION_ERROR(-20004, 'Primary key violation: ENO must be unique');
    END IF;
END;
/
```

Question 10

Write a trigger to implement a foreign key constraint on the DEPTNO column of the EMP table which refers to the DEPTNO of DEPT Table

```
CREATE OR REPLACE TRIGGER trg_emp_foreign_key
BEFORE INSERT OR UPDATE OF DEPTNO ON EMP
FOR EACH ROW
DECLARE
    v_count NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_count FROM DEPT WHERE DEPTNO = :NEW.DEPTNO;
    IF v_count = 0 THEN
        RAISE_APPLICATION_ERROR(-20005, 'Foreign key violation: DEPTNO must exist in DEPT
table');
    END IF;
END;
/
```